



Universidad de San Andrés

Big Data

Trabajo Práctico 3

Otoño 2026

Integrantes:

Constanza María Efkhanian
Julián Fabrizio Notario
María Florencia Pascale

Fecha de entrega: 15 de mayo de 2026

A. Enfoque de validación

1. Realizado en *Notebook*.

2. A continuación, se encuentra la tabla de la diferencia de medias entre ocupados para los años 2005 y 2025:

Table 1: Diferencia de medias entre ocupados 2005 y 2025

Variable	Media 2005	Media 2025	Diferencia	p-valor	Sig.
Edad	39.93	42.52	2.59	0.000	***
Años de educación	8.71	9.71	1.01	0.000	***
Ingreso familiar	2,576,031	2,298,457	-277,574	0.000	***
Personas en el hogar	3.86	3.46	-0.40	0.000	***
Horas trabajadas	39.17	35.88	-3.29	0.000	***

En primer lugar, algo llamativo que se puede apreciar en la tabla es que todas las diferencias de medias son estadísticamente significativas. En particular, se puede observar un claro incremento en la edad promedio de los trabajadores, con una diferencia de casi tres años entre 2005 y 2025. Además de esto, también se puede ver que hubo una mejora en el nivel educativo de los ocupados, de aproximadamente un año más, lo cual indica que la fuerza laboral está más calificada en 2025 respecto de la de 2005. Por otro lado, se puede ver una reducción en el ingreso familiar promedio entre 2005 y 2025, así como en la cantidad de personas en el hogar, lo cual puede ser un reflejo de la caída en la natalidad en Argentina. Por último, se puede observar una caída considerable en la cantidad de horas promedio trabajadas en 2025, con una reducción de aproximadamente 3 horas respecto al 2005.

3. Realizado en *Notebook*.

B. Modelo de Regresión Logística

1. A continuación, se encuentra la tabla de la regresión logística:

Table 2: Regresión Logística: coeficientes, errores estándar (SE) y odd-ratios (OR) (2005 vs 2025)

	Coef. 2005	SE 2005	OR 2005	Sig. 2005	Coef. 2025	SE 2025	OR 2025	Sig. 2025
Intercepto	3.999	0.391	54.541	***	4.874	0.674	130.847	***
Edad	-0.111	0.017	0.895	***	-0.192	0.029	0.825	***
Edad ²	0.001	0.000	1.001	***	0.002	0.000	1.002	***
Años de educación	-0.128	0.014	0.880	***	-0.115	0.024	0.891	***
Ingreso total familiar	0.000	0.000	1.000	***	0.000	0.000	1.000	***
Personas en el hogar	0.057	0.023	1.059	**	0.100	0.051	1.105	**

Primero, podemos observar que el coeficiente para "Edad" es negativo y el de "Edad²" es positivo, lo que indica que la relación entre edad y probabilidad de estar ocupado sigue un tipo de forma de U invertida. Es decir, la probabilidad de estar ocupado crece con la edad hasta un punto máximo, y luego decrece, lo cual tiene sentido porque a partir de los 60-65 años, aproximadamente, las personas comienzan a jubilarse.

Dep.
Variable:
informal

Además, algo sorprendente a primera vista es que el coeficiente de años de educación es negativo para ambos años, y el *odds-ratio* es menor que 1, lo que indica que más educación reduce las chances de estar ocupado, aunque esto se puede interpretar como que más educación está asociada a seguir estudiando en vez de trabajar.

Otro dato llamativo es que los ingresos totales familiares presentan un coeficiente prácticamente de 0 en los años 2005, como 2025, con un *odds-ratio* de 1, lo que implica potencialmente que el ingreso familiar no tiene un efecto sobre el estado de ocupación.

Adicionalmente, el coeficiente de la variable de "Personas en el hogar" es positivo para ambos años, con un *odds-ratio* mayor que 1, lo que indica que un mayor nivel de personas en el hogar aumenta las probabilidades de estar ocupado, con un efecto más fuerte en 2025 que en 2005.

2. El siguiente gráfico muestra la probabilidad de ser informal con respecto al logaritmo del ingreso familiar. Se eligió representar la variable en logaritmos con el fin de poder visualizar mejor la curva S:

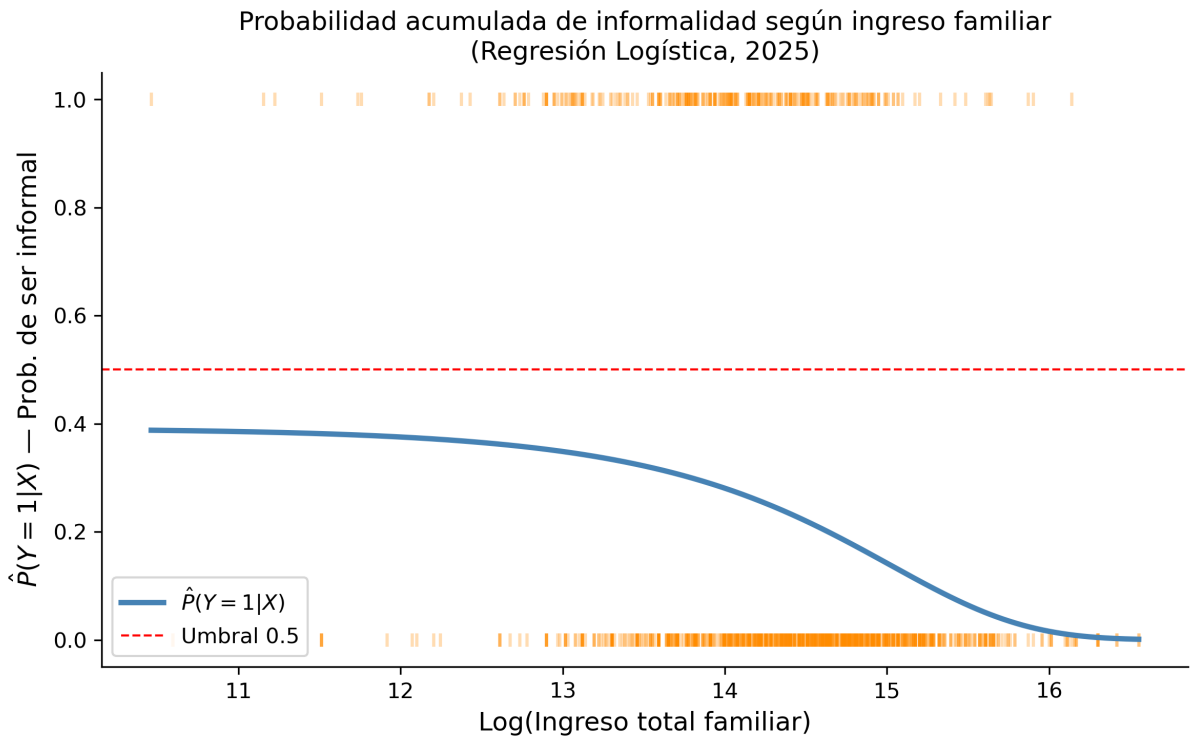


Figure 1: Probabilidad de ser informal dependiendo del ingreso total familiar (en logaritmos).

Como se observa, la curva presenta la forma de S típica de la regresión logística, solo que invertida. Esto tiene sentido para la variable elegida; a medida que aumenta el ingreso familiar, la probabilidad de informalidad cae de manera sostenida. Para ingresos bajos, la probabilidad ronda el 35-40%, mientras que para ingresos altos, cae por debajo del 5%. Por último, cabe aclarar que la curva nunca cruza el umbral de 0.5, lo que sugiere que el ingreso por sí solo no es suficiente para clasificar a una observación como "informal".

C. Método de Vecinos Cercanos (KNN)

3. Esencialmente, un K grande implica una varianza menor, puesto que suaviza las predicciones al promediar con más vecinos, pero al costo de un mayor sesgo, lo que resulta en un *underfitting* del modelo. En cambio, un número pequeño de K resulta en un menor nivel de sesgo, dado que el modelo ajusta mucho a los datos de la muestra, pero al costo de una mayor varianza, lo que resulta en un *overfitting* del modelo.

Table 3: Accuracy en train y test para KNN con $K=\{10, 50, 100\}$ para 2005 y 2025

K	2005 Train	2005 Test	2025 Train	2025 Test
10	0.745	0.721	0.795	0.784
50	0.728	0.721	0.760	0.776
100	0.725	0.720	0.756	0.771

Se observa que a medida que K aumenta, la accuracy en train cae en ambos años, consistente con el mayor sesgo que introduce un K grande. En 2025 la accuracy en test es mayor que en 2005 para todos los K , mientras que en 2005 los resultados prácticamente no varían con la elección de K .

4. Lo siguiente es el gráfico de KNN de edades vs. horas trabajadas.

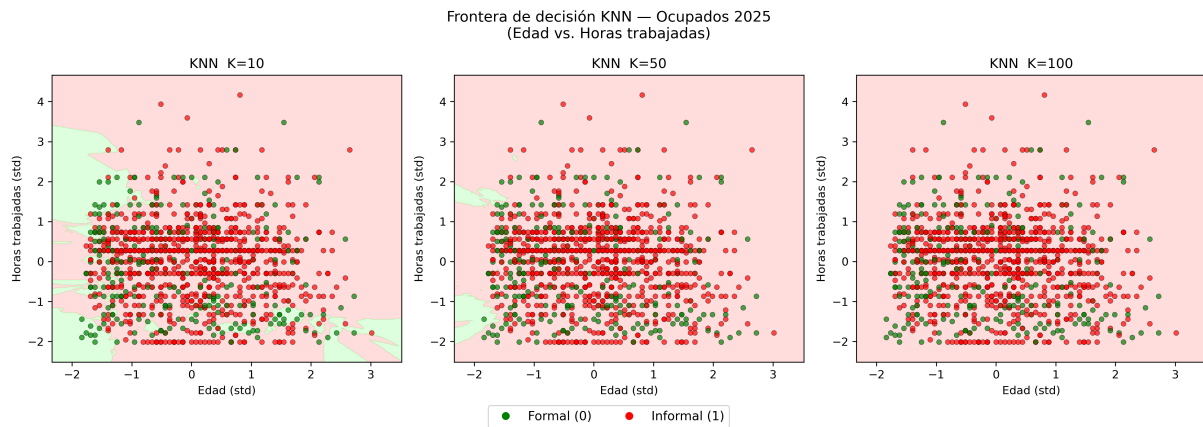


Figure 2: Frontera de decisión KNN. Ocupados 2025 (Edad vs. Horas trabajadas). El fondo rojo indica la región predicha como informal, y el verde, como formal.

Hay un error en los
colores-> Informal:
verde; formal: rojo.
Coincide con la idea de
"hay mas formales"

5. Selección del K óptimo por Cross-Validation.

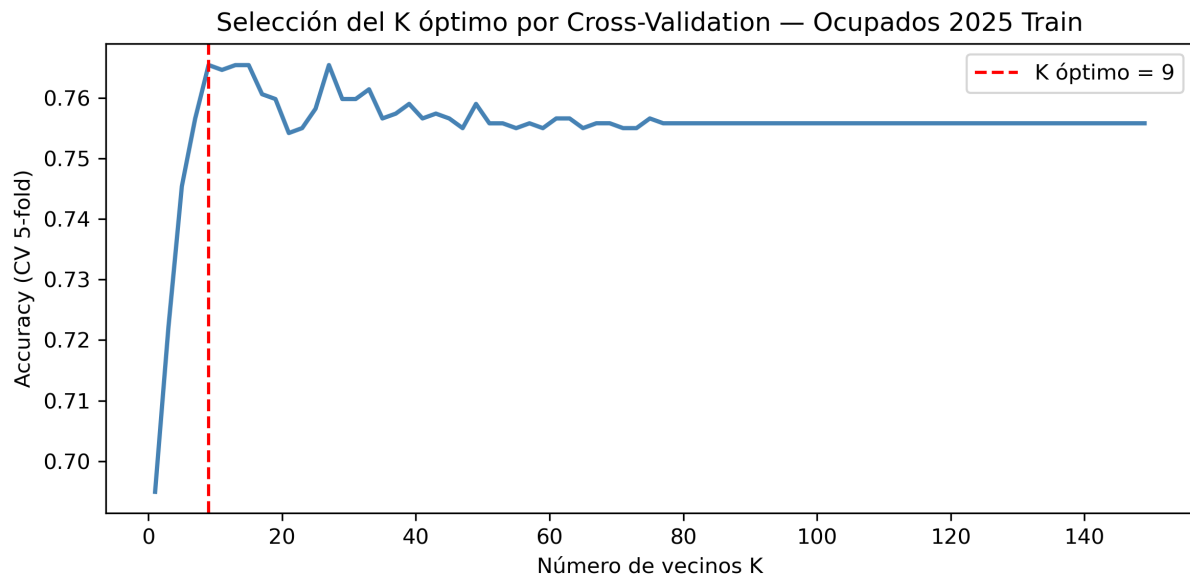


Figure 3: Selección del K óptimo por Cross-Validation — Ocupados 2025 Train.

La curva muestra la *accuracy* promedio de CV 5-fold para cada K impar entre 1 y 149. El K óptimo seleccionado es K=9 (línea roja punteada), con una *accuracy* promedio de CV de 0.765 y una *accuracy* en el set de test de 0.776.

D. Desempeño de modelos, elección y predicción afuera de la muestra

6. A continuación, se muestran las curvas ROC y la matriz de confusión. Las métricas elegidas son *accuracy* y *recall*.

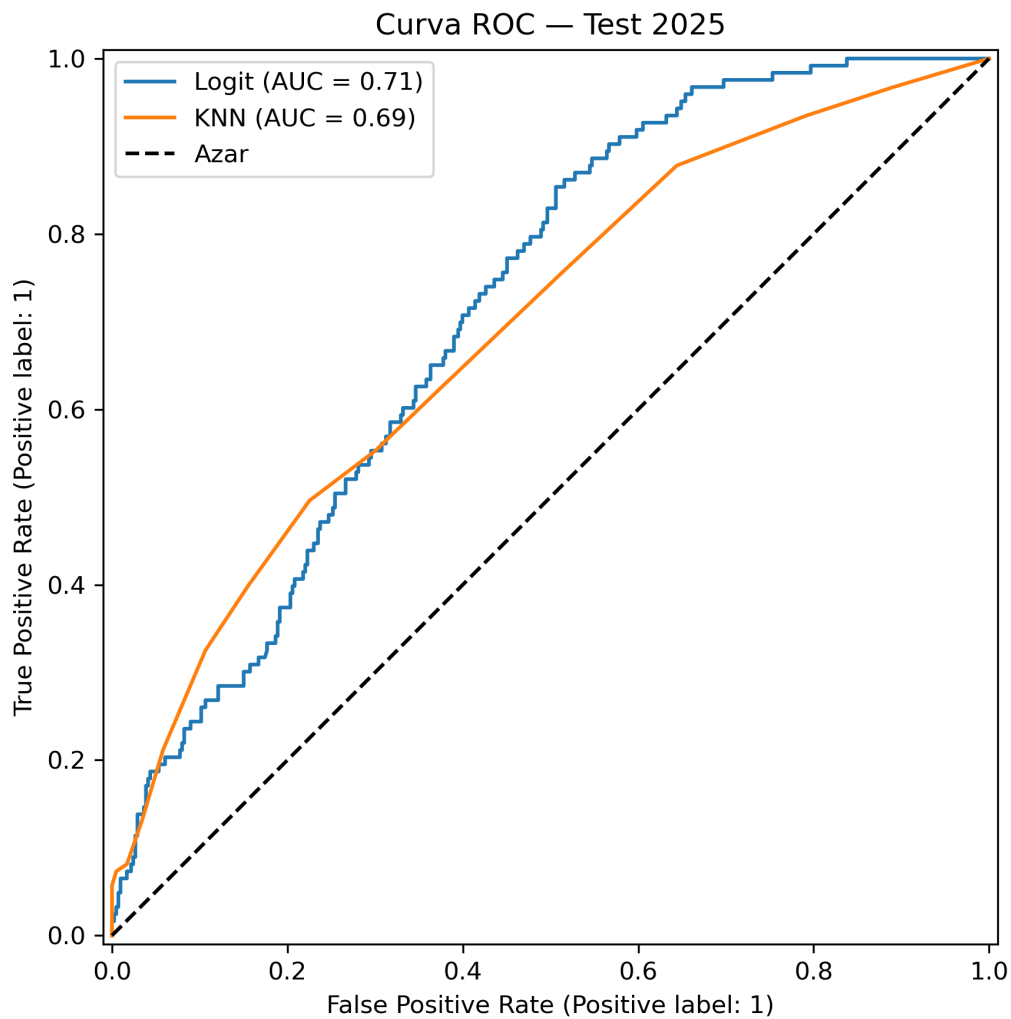


Figure 4: Curvas ROC de ambos modelos.

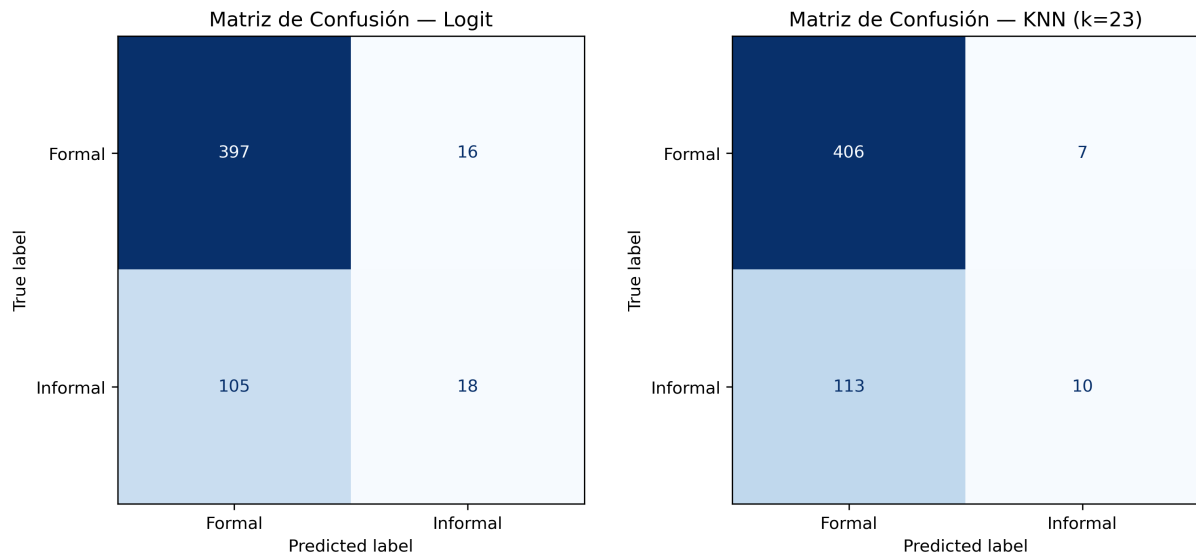


Figure 5: Matriz de confusión para ambos modelos.

Table 4: Comparación de métricas: Logit vs. KNN

Métrica	Logit	KNN
Accuracy	0.77	0.78
Recall	0.15	0.08
AUC-ROC	0.71	0.69

Se comparó el desempeño del modelo logit y el KNN con K óptimo ($K=23$) sobre la base de testeo de 2025. En términos de *accuracy*, ambos modelos presentan un desempeño similar. No obstante, esta métrica resulta engañosa dado el desbalance de clases en la muestra: como la mayoría de los ocupados son formales, un modelo que prediga siempre "formal" alcanzaría un *accuracy* elevado sin detectar ningún informal. El *recall* expone esta limitación: el logit identifica correctamente el 15% de los trabajadores informales reales, mientras que el KNN apenas alcanza el 8%. Vale decir que ambos valores son bajos, pero el logit duplica al KNN en la detección de verdaderos positivos. Finalmente, la curva ROC confirma la leve superioridad del logit, indicando una mejor capacidad de discriminación entre clases a lo largo de todos los umbrales posibles.

muy bien!

7. Para este problema, el modelo más adecuado es el logit. Dado que se busca identificar trabajadores precarizados para asignarles recursos, el error más costoso es el de tipo II: clasificar a un informal como formal y que este no reciba ayuda. Por esto, la métrica importante es *recall*, que mide la proporción de los verdaderos informales que son correctamente identificados. El logit tiene un *recall* de 0.15 mientras que el KNN tiene uno de

0.08; es decir, el logit captura casi el doble de los informales que el KNN.

En términos del trade-off, si bien reducir el umbral de clasificación por debajo de 0.5 incrementaría los falsos positivos, aumentaría el *recall* y reduciría las exclusiones indebidas.

compatibilidad
entre la
biblioteca y la
version de la
computadora?

E. Herramientas de IA y reflexión final

1. LA IA nos sirvió principalmente como guía sobre las funciones que debíamos usar y para solucionar errores de código y bibliotecas. Hubo una excepción: un problema de compatibilidad con la biblioteca *statsmodels* que la IA no logró resolver, lo cual se solucionó reasignando internamente las tareas del grupo para que el integrante afectado trabajara en las secciones que no requerían dicha biblioteca. Para las interpretaciones, optamos por no usar IA para asentar los conocimientos adquiridos en la materia. **muy bien!**

2. A continuación, se muestran las *screenshots* de las interacciones con AI (todos los integrantes utilizamos Claude AI):



Dame una lista de los códigos que necesitaría utilizar para completar estos incisos. Incluí la función de cada código.

Catalogó códigos Python necesarios con funciones descriptivas >

Acá te dejo los códigos necesarios para cada inciso:

Inciso 1 — Separar la base por año

```
python
ocupados_2005 = ocupados[ocupados['año'] == 2005]
ocupados_2025 = ocupados[ocupados['año'] == 2025]
```

Filtra el DataFrame según el valor de la columna de año.

Inciso 2 — Tabla de diferencia de medias

```
python
# Calcular medias por grupo
medias_2005 = ocupados_2005[variables_X].mean()
medias_2025 = ocupados_2025[variables_X].mean()
```

Calcula la media de cada variable seleccionada en cada submuestra.

```
python
from scipy import stats

# T-test para cada variable
for var in variables_X:
    t, p = stats.ttest_ind(ocupados_2005[var], ocupados_2025[var], nan_policy='omit')
```

ttest_ind realiza un test t de diferencia de medias entre dos grupos independientes y devuelve el estadístico t y el p-valor.

```
python

# Armar tabla resumen
import pandas as pd
tabla = pd.DataFrame({'Media 2005': medias_2005, 'Media 2025': medias_2025,
                      'Diferencia': medias_2025 - medias_2005,
                      'p-valor': [p_values]})
```

Construye la tabla comparativa con las medias, la diferencia y la significancia estadística.

Inciso 3 — Train/Test Split para cada año

```
python

from sklearn.model_selection import train_test_split

# Agregar columna de unos para el intercepto
ocupados_2005['const'] = 1
ocupados_2025['const'] = 1
```

Añade la constante necesaria para estimar el intercepto en la regresión.

```
python

# Definir X e y para cada año
X_2005 = ocupados_2005[variables_X + ['const']]
y_2005 = ocupados_2005['informal']

X_2025 = ocupados_2025[variables_X + ['const']]
y_2025 = ocupados_2025['informal']
```

Separa la variable dependiente (`informal`) de las independientes (matriz X).

```
python

# Split para 2005
X_train_05, X_test_05, y_train_05, y_test_05 = train_test_split(
    X_2005, y_2005, test_size=0.30, random_state=444)

# Split para 2025
X_train_25, X_test_25, y_train_25, y_test_25 = train_test_split(
    X_2025, y_2025, test_size=0.30, random_state=444)
```

`train_test_split` divide aleatoriamente los datos en 70% entrenamiento y 30% prueba.
`random_state=444` fija la semilla para reproducibilidad.


```
python

import statsmodels.api as sm

modelo_sm = sm.Logit(y_train, X_train).fit()
modelo_sm.summary()
modelo_sm.bse # Errores estándar
```

Alternativa con `statsmodels`, que devuelve directamente errores estándar, p-valores y una tabla de resultados completa.

```
python

tabla = pd.DataFrame({
    'Coeficiente': ...,
    'Error Estándar': ...,
    'Odd-Ratio': ...
})
```

Arma la tabla comparativa entre ambos años.

Inciso 2 — Visualización

```
python

y_prob = modelo.predict_proba(X_test)[: , 1]
```

Obtiene la probabilidad predicha $P(Y=1|X)$ para cada observación.

```
python

import matplotlib.pyplot as plt

plt.scatter(X_test['variable_numerica'], y_prob)
plt.xlabel('Variable numérica')
plt.ylabel('P(informal = 1)')
plt.title('Probabilidad de ser informal')
plt.show()
```

Grafica la probabilidad predicha en función de una variable numérica.

```
python

# Para mostrar la probabilidad acumulada ordenando por la variable
df_plot = pd.DataFrame({'var': X_test['variable_numerica'], 'prob': y_prob})
df_plot = df_plot.sort_values('var')
plt.plot(df_plot['var'], df_plot['prob'].cumsum() / df_plot['prob'].cumsum().max())
```

Decime las funciones que necesito usar para el C.3, como se usan (parametros, formatos aceptados) y qué hacen teoricamnete.

Las funciones usadas en C.3 son dos:

`StandardScaler()` — de `sklearn.preprocessing`

Estandariza las features restando la media y dividiendo por el desvío estándar, de modo que cada variable quede con media 0 y varianza 1. Esto es necesario para KNN porque el algoritmo mide distancias entre observaciones: si una variable tiene escala mucho mayor que otra (ej. ingreso en millones vs edad en años), va a dominar el cálculo de distancia y sesgar los resultados.

Se usa en dos pasos:

- `.fit_transform(X_train)` → aprende la media y desvío **del train** y transforma. Devuelve un array numpy.
- `.transform(X_test)` → aplica la **misma** transformación aprendida del train al test. Nunca se hace fit sobre el test para no filtrar información.

`KNeighborsClassifier(n_neighbors=k)` — de `sklearn.neighbors`

Implementa el clasificador KNN. Para clasificar una observación nueva, busca las K observaciones más cercanas en el espacio de features (por distancia euclídea por defecto) y asigna la clase mayoritaria entre esos K vecinos. K chico → frontera irregular, alta varianza. K grande → frontera suave, alto sesgo.

Parámetros relevantes:

- `n_neighbors` : cantidad de vecinos a considerar (entero, obligatorio)
- `metric` : distancia a usar (default `'minkowski'` con $p=2$, equivalente a euclídea)

Decime las funciones que necesito usar para el C.4, como se usan (parametros, formatos aceptados) y qué hacen teorícamnete.

Las funciones usadas en C.4 son:

`ListedColormap(['#FFAAAA', '#AAFFAA'])` — de `matplotlib.colors`

Crea un mapa de colores discreto a partir de una lista de colores. Se usa para pintar el fondo del gráfico según la clase predicha: el primer color corresponde a la clase 0 (formal) y el segundo a la clase 1 (informal). Acepta nombres de colores (`'red'`) o códigos hex (`'#FFAAAA'`).

`np.meshgrid(x, y)` — de `numpy`

Crea una grilla rectangular de puntos en 2D a partir de dos vectores. Se le pasan dos arrays 1D (uno para cada eje) y devuelve dos matrices `xx` e `yy` que juntas representan todas las combinaciones de coordenadas de la grilla. Se usa para generar los puntos sobre los que el modelo va a predecir y así poder dibujar la frontera de decisión.

python

```
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                    np.linspace(y_min, y_max, 300))
```

`np.c_[xx.ravel(), yy.ravel()]` — de `numpy`

`ravel()` aplanar cada matriz en un vector 1D. `np.c_` concatena esos dos vectores como columnas, formando un array de shape `(N, 2)` que representa todos los puntos de la grilla. Ese array es el que se le pasa al `.predict()` de `...NN`.

Decime las funciones que necesito usar para el C.5, como se usan (parametros, formatos aceptados) y qué hacen teoricamnete.

Una sola función nueva respecto a lo anterior:

`cross_val_score(estimator, X, y, cv, scoring)` — de `sklearn.model_selection`

Evalúa un modelo usando cross-validation. Divide el conjunto de datos en `cv` partes (folds), entrena el modelo en `cv-1` partes y evalúa en la parte restante, rotando hasta que cada parte haya sido usada como validación una vez. Devuelve un array numpy con la métrica calculada en cada fold.

Parámetros:

- `estimator`: el modelo a evaluar, en este caso `KNeighborsClassifier(n_neighbors=k)`. No hace falta haberlo entrenado antes, `cross_val_score` hace el fit internamente.
- `X`: matriz de features. Acepta array numpy o DataFrame, debe estar ya escalado.
- `y`: vector de etiquetas. Array 1D o Series de pandas.
- `cv=5`: número de folds. Con 5 divide el train en 5 partes iguales.
- `scoring='accuracy'`: métrica a calcular en cada fold. Otras opciones serían `'recall'`, `'f1'`, `'roc_auc'`, etc.

Devuelve un array de 5 valores (uno por fold). Para obtener la accuracy promedio se le aplica `.mean()` y para la variabilidad `.std()`.

```
python
scores = cross_val_score(KNeighborsClassifier(n_neighbors=k),
                        X_2025_train_scaled, y_2025_train,
                        cv=5, scoring='accuracy')
scores.mean() # accuracy promedio entre los 5 folds
```

Dame las funciones que necesito usar el ejercicio 6, como se usan y que hacen teoricamente

1:03 p.m.    < 2/2 >

Compiló contexto para explicar funciones del ejercicio >

Acá tenés un resumen completo de todas las funciones que necesitás para el ejercicio 6, basado exactamente en lo que usaron en el tutorial:

Imports necesarios

```
python
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, KFold
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             recall_score, RocCurveDisplay, roc_auc_score)
import matplotlib.pyplot as plt
```


1. Modelos

```
LogisticRegression(penalty=None).fit(X_train, y_train)
```

Teoría: modela $P(Y=1|X)$ usando la función sigmoide: $P = 1/(1+e^{-X\beta})$. Los coeficientes β se estiman por máxima verosimilitud. La frontera de decisión es lineal. `penalty=None` significa sin regularización.

```
KNeighborsClassifier(n_neighbors=k)
```

Teoría: para una observación nueva, busca los k vecinos más cercanos en el espacio de features (por distancia euclidiana) y clasifica por mayoría de votos. No hay función paramétrica: el modelo "memoriza" los datos de entrenamiento. Con k chico es muy flexible (riesgo de overfitting), con k grande es más suave.

2. Predicciones

```
python

# Clases predichas (umbral 0.5 por defecto)
y_pred_logit = logit.predict(X_test)
y_pred_knn = knn_best.predict(X_test)

# Probabilidades (necesarias para ROC)
y_prob_logit = logit.predict_proba(X_test)[: , 1]
y_prob_knn = knn_best.predict_proba(X_test)[: , 1]
```

`predict_proba(X)[: , 1]` devuelve $P(Y=1|X)$ para cada observación. La columna `[: , 1]` selecciona la probabilidad de la clase positiva (informal=1).

3. Matriz de confusión

```
python

cm = confusion_matrix(y_test, y_pred_logit)
tn, fp, fn, tp = cm.ravel()
```

Teoría: tabla 2×2 que cruza clases reales vs. predichas. Con umbral $p > 0.5$:

4. Métricas

```
python

# Accuracy: (TP+TN)/(TP+TN+FP+FN)
accuracy_score(y_test, y_pred_logit)

# Recall / Sensibilidad: TP/(TP+FN) - ¿qué % de informales detectamos?
recall_score(y_test, y_pred_logit)

# AUC-ROC (métrica resumen de la curva)
roc_auc_score(y_test, y_prob_logit)
```

Abajo de eso, simplemente agregás el bloque del gráfico. Quedaría así todo junto:

```
python

# Matrices de confusión (valores numéricos)
cm_logit = confusion_matrix(y_2025_test, y_pred_logit)
tn, fp, fn, tp = cm_logit.ravel()

cm_knn = confusion_matrix(y_2025_test, y_pred_knn)

# Visualización
from sklearn.metrics import ConfusionMatrixDisplay

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

ConfusionMatrixDisplay.from_estimator(
    logit_sk, X_2025_test_scaled, y_2025_test,
    display_labels=['Formal', 'Informal'],
    colorbar=False, ax=axes[0]
)
axes[0].set_title('Matriz de Confusión - Logit')

ConfusionMatrixDisplay.from_estimator(
    knn_kcv, X_2025_test_scaled, y_2025_test,
    display_labels=['Formal', 'Informal'],
    colorbar=False, ax=axes[1]
)
axes[1].set_title(f'Matriz de Confusión - KNN (k={k_optimo})')

plt.tight_layout()
plt.savefig('matrices_confusion_2025.png', dpi=300, bbox_inches='tight')
plt.show()
```

excerpt_from_previous_claude_message.txt

10 líneas

TXT

aca en metricas me tira un solo numero, no debería ser dos?

El problema es que Jupyter solo muestra el resultado de la última línea de la celda. Las dos primeras (`accuracy_score` y `recall_score`) no se imprimen porque no tienen `print()`.

Reemplazá todo por esto:

```
python

print(f"Accuracy  Logit: {accuracy_score(y_2025_test, y_pred_logit):.4f}")
print(f"Recall    Logit: {recall_score(y_2025_test, y_pred_logit):.4f}")
print(f"AUC-ROC   Logit: {roc_auc_score(y_2025_test, y_prob_logit):.4f}")

print()

print(f"Accuracy  KNN:  {accuracy_score(y_2025_test, y_pred_knn):.4f}")
print(f"Recall    KNN:  {recall_score(y_2025_test, y_pred_knn):.4f}")
print(f"AUC-ROC   KNN:  {roc_auc_score(y_2025_test, y_prob_knn):.4f}")
```

3. Aproximadamente, transcurrieron unas siete horas desde el comienzo del trabajo práctico hasta su finalización.